

MNIST Handwritten Digit Classification with Convolutional Neural Networks

AI 100 -Midterm Project Report

Kreesh Patel

GitHub: github.com/kreeshpatel/ai100-mnist-cnn-classification

1. Problem Definition and Dataset Curation

The goal of this project is to build a deep learning model that can accurately classify handwritten digits (0 through 9) from grayscale images. Handwritten digit recognition is a foundational problem in computer vision and serves as an excellent benchmark for evaluating neural network architectures.

Dataset: MNIST

The MNIST (Modified National Institute of Standards and Technology) dataset is one of the most widely used benchmarks in machine learning. It was originally compiled by Yann LeCun and colleagues and contains handwritten digits collected from Census Bureau employees and high school students.

Training set: 60,000 images

Test set: 10,000 images

Image size: 28 x 28 pixels, single-channel grayscale

Classes: 10 (digits 0-9)

Preprocessing

Each image is converted to a PyTorch tensor and normalized using the dataset's global mean (0.1307) and standard deviation (0.3081). This normalization centers pixel values around zero and helps the network converge faster during training. No data augmentation was applied, as the standard MNIST benchmark does not typically require it to achieve high accuracy.

2. Deep Learning Model

The model is a Convolutional Neural Network (CNN) implemented in PyTorch. CNNs are the standard architecture for image classification tasks because their convolutional layers can automatically learn spatial features such as edges, curves, and shapes directly from raw pixel data.

Architecture (MNISTNet)

```
Input (1 x 28 x 28)
-> Conv2d(1, 32, kernel=3, padding=1) -> ReLU -> MaxPool(2)    [32x14x14]
-> Conv2d(32, 64, kernel=3, padding=1) -> ReLU -> MaxPool(2)  [64x7x7]
-> Flatten                                                    [3136]
-> Linear(3136, 128) -> ReLU
-> Linear(128, 10)                                           [10 logits]
```

The first convolutional layer extracts 32 low-level feature maps (edges, simple patterns). The second convolutional layer builds 64 higher-level feature maps from those. Each convolutional layer is followed by a ReLU activation for non-linearity and a 2x2 max-pooling layer that halves the spatial dimensions and provides translation invariance. The output is flattened into a 3,136-dimensional vector and passed through two fully connected layers, producing 10 raw logits (one per digit class).

Training Configuration

Loss function: CrossEntropyLoss (combines LogSoftmax + NLLLoss)

Optimizer: Adam (lr = 0.001)

Batch size: 64

Epochs: 5

Random seed: 42 (for reproducibility)

Device: CPU

Adam was chosen as the optimizer because it adapts learning rates per parameter, which generally leads to faster convergence than standard SGD on this type of task. CrossEntropyLoss is the standard choice for multi-class classification problems.

3. Results

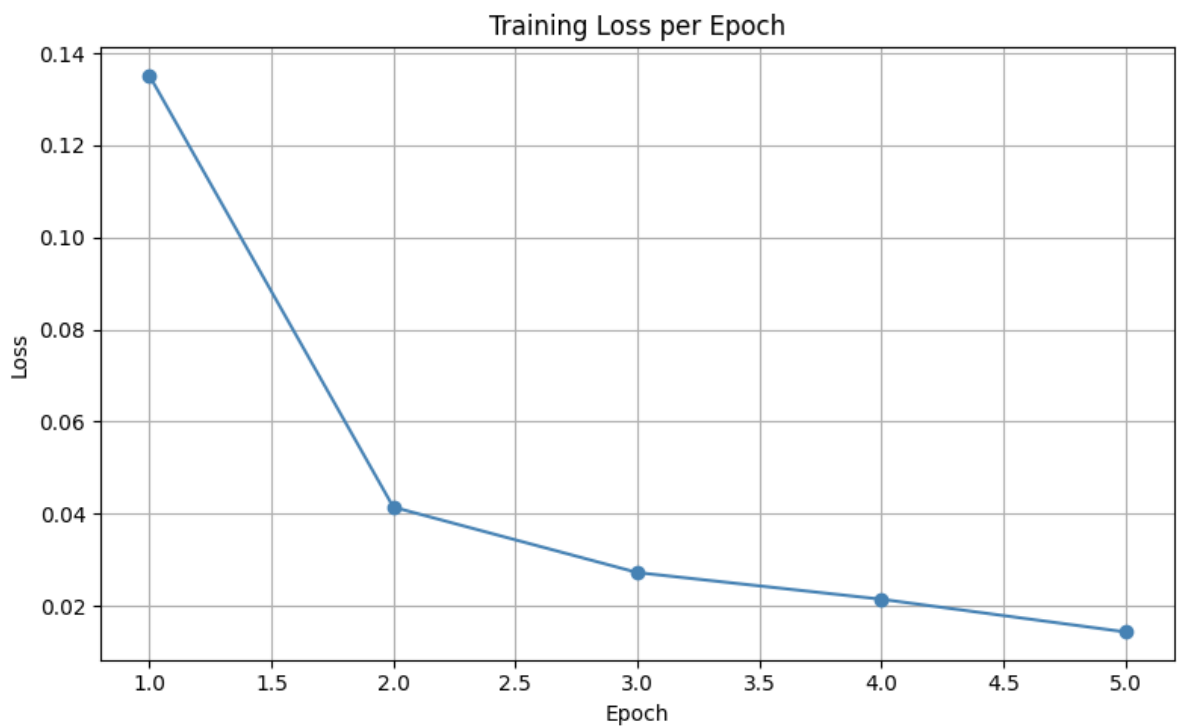
Training Performance

Epoch	Training Loss	Training Accuracy
1	0.1352	95.82%
2	0.0415	98.71%
3	0.0273	99.17%
4	0.0215	99.30%
5	0.0144	99.54%

Final Test Accuracy: 99.02%

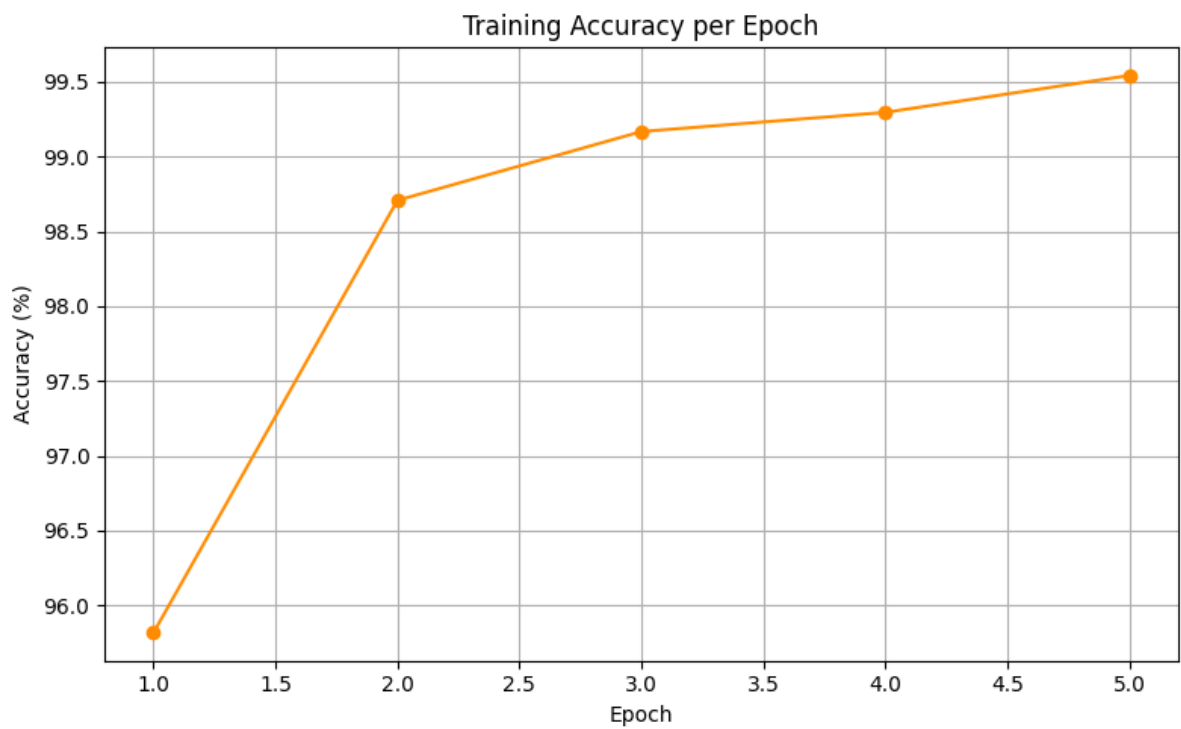
The model converges rapidly, achieving over 95% training accuracy after just the first epoch. By epoch 5, training accuracy reaches 99.54% and the test accuracy is 99.02%, demonstrating strong generalization with minimal overfitting.

Training Loss Curve



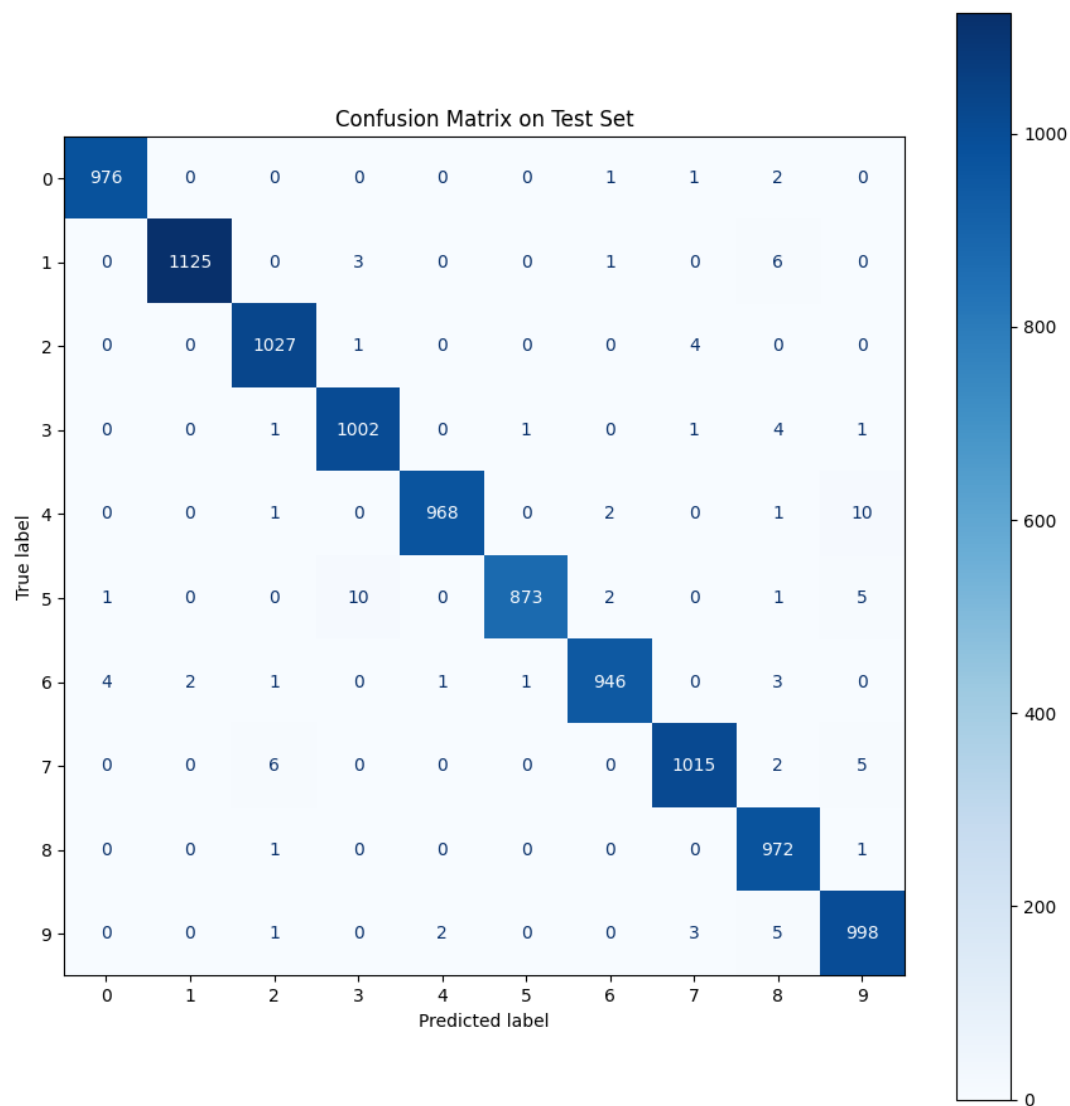
The loss decreases sharply after the first epoch and continues to decline steadily, indicating effective learning without signs of divergence.

Training Accuracy Curve



Training accuracy climbs quickly to above 98% by epoch 2 and plateaus near 99.5% by epoch 5, showing the model has effectively learned the digit patterns.

Confusion Matrix (Test Set)



The confusion matrix shows the model classifies nearly all test digits correctly. The diagonal is dominant with very few off-diagonal misclassifications. The most common errors involve visually similar digit pairs such as 4/9 and 3/5, which is expected and consistent with human confusion patterns.

4. Lessons and Experience Learned

Key Takeaways

- **CNNs are well-suited for image tasks:** Convolutional layers automatically learn spatial features (edges, curves, digit shapes) without manual feature engineering. This is a major advantage over traditional machine learning approaches that require hand-crafted features.
- **Data normalization matters:** Normalizing inputs to zero mean and unit variance helped the model train faster and more stably. Without normalization, gradient magnitudes can vary wildly across features.
- **Simple architectures can be highly effective:** A two-layer CNN with only ~400K parameters achieved 99.02% test accuracy on MNIST. This demonstrates that more complex models are not always necessary -choosing the right architecture for the problem is more important than adding layers.
- **Reproducibility is important:** Setting a random seed ensured consistent results across runs, making it easier to compare experiments and debug issues during development.
- **PyTorch provides a clean workflow:** The combination of Dataset/DataLoader for data handling, nn.Module for model definition, and autograd for backpropagation made the end-to-end implementation straightforward.

Future Improvements

- Add dropout layers to reduce overfitting on more complex datasets.
- Experiment with data augmentation (rotation, scaling) to improve robustness.
- Try deeper architectures or batch normalization for potentially higher accuracy.
- Extend the project to harder datasets like Fashion-MNIST or CIFAR-10.